Plan de trabajo del MVP

Vía Justa · app móvil para que nadie llegue a una ciudad colombiana sin saber qué es un precio o un contrato justo

Versión 1.0 · 19 de octubre de 2026 · Equipo de 2 personas (frontend y backend)
Base: lineamientos v3, mockups de Figma, requisitos v0.1 y decisiones D1–D10

1. Resumen ejecutivo

Qué se construye. Una app Android hecha en Flutter cuyo núcleo es el **semáforo de justicia**: compara un precio contra reportes comunitarios (regla estadística) y analiza cláusulas de contratos con IA. Alrededor del núcleo hay un mapa de Colombia con una ciudad piloto (Valledupar), una vista de ciudad con cultura local y el registro de negocios por parte de los usuarios.

Cuánto tiempo hay. Seis semanas hasta finales de noviembre de 2026 y un **primer avance en pocos días**. Son dos personas: tú haces el frontend y tu compañero el backend. La fecha exacta del avance y de la entrega final están por confirmar.

Cómo se aborda. El avance consiste en **convertir los mockups de Figma en pantallas Flutter en VS Code**, con datos de ejemplo, sin esperar al backend. Después, semana a semana, cada pantalla se conecta a datos reales en este orden: cuenta y mapa, precios y semáforo, contratos, negocios. Todo lo que no cabe se mueve a “si hay tiempo” o a Fase 2 de forma explícita.

> **Mensaje clave del avance:** mostrar un flujo completo y navegable en un teléfono Android real (entrada, onboarding, mapa, municipio piloto, inicio, análisis con resultado de semáforo, idioma español/inglés), dejando claro qué está implementado, qué es prototipo y qué está planificado.

Estado de las decisiones en una vista

Acordado	Propuesto (falta confirmar)	Pendiente de definir
Flutter solo Android	D4: Supabase (compañero de backend)	Fecha del avance y de la entrega
Idiomas es/en con easy_localization	D8: arquitectura por capas ligeras y agente de código con archivo de reglas	Facturación de Google Cloud
Google Maps	Barra inferior fija de 5 pestañas	P6 tarifa oficial como tope
Riverpod y go_router	Código en inglés, documentación en español	P9, P10, P11
Gemini para contratos	Solo Valledupar como piloto (P8)	Lista de categorías de precio
Negocios sin pedidos internos; contacto por WhatsApp		Nombre de paquete Android
GitHub con ramas por tarea y PR		Fuente del mapa SVG
Reglas del semáforo (mediana, 15 %/40 %, N = 3, 60 días, aviso por precio bajo, evento)		Horas disponibles por persona
Formulario de negocio en 4 pasos		Política del curso sobre IA

Convención de todo el documento: **[Acordado]** es una decisión tomada; **[Propuesto]** es una recomendación que aún debes confirmar; **[Pendiente]** es algo sin definir, con un valor por defecto sugerido en la sección 10.

2. Punto de partida y decisiones técnicas

2.1 Hechos confirmados

- No hay código. Existen mockups en Figma (17 pantallas), documentación UML y el documento de lineamientos v3.
- Nivel de Flutter básico; ya se usó Supabase y otras herramientas de backend. El curso no limita las herramientas.
- Feedback del profesor: vista de ciudad con cultura y oferta local, registro de negocios por parte de los usuarios y uso de Google Maps.
- Los requisitos están en un documento aparte (v0.1): 42 requisitos funcionales, 10 reglas de negocio, 13 historias de usuario y 11 requisitos no funcionales.

2.2 Registro de decisiones

ID	Decisión	Estado	Efecto en el trabajo
D1	MVP en español e inglés	[Acordado]	Todo texto va por traducciones desde el primer día
D2	easy_localization para la interfaz	[Acordado]	Archivos es.json y en.json con las mismas claves
D3	Google Maps (google_maps_flutter) y Places	[Acordado]	Requiere clave y cuenta de facturación de Google Cloud
D4	Supabase: PostgreSQL, PostGIS, Auth, Storage y Edge Function	[Propuesto]	Bloquea el contrato de datos; confirmar con el compañero de backend
D5	Riverpod y go_router	[Acordado]	Estado por funcionalidad y rutas en un solo archivo
D6	Negocios sin pedidos internos: el pedido va a un enlace externo	[Acordado]	Campo opcional “enlace de pedido” por negocio
D7	Contacto por WhatsApp, sin chat propio	[Acordado]	Enlace wa.me con mensaje prellenado en el idioma del usuario
D8	Arquitectura por capas ligeras y agente de código con archivo de reglas	[Propuesto]	Define la estructura de carpetas y cómo se usa la IA al programar
D9	Gemini como proveedor del LLM	[Acordado]	Clave solo en la Edge Function; con nivel gratuito, solo contratos de prueba
D10	Un repositorio, ramas cortas por tarea y PR hacia main	[Acordado]	main protegida; revisión del otro integrante
—	Solo Android en el MVP; VS Code como editor	[Acordado]	Se crea el proyecto con --platforms=android

2.3 Herramientas

Capa	Herramienta	Cuándo entra
App	Flutter/Dart, flutter_riverpod, go_router, easy_localization, flutter_svg	Avance (día 1)
App	google_maps_flutter, url_launcher, supabase_flutter, ML Kit de reconocimiento de texto, selector de imágenes/archivos y de ubicación (paquetes por elegir)	Semanas 2 a 5, cuando la tarea los necesite
Backend	Supabase (PostgreSQL, PostGIS, RLS, Auth, Storage, Edge Functions), Supabase CLI y Docker para entorno local	Semana 1 (esquema) y siguientes
IA	API de Gemini invocada desde la Edge Function	Semana 4
Calidad	flutter_test, mocktail, GitHub Actions (analyze y test)	Desde el avance
Diseño y docs	Figma, PlantUML, Markdown en docs/	Continuo

Los paquetes se agregan cuando la tarea los necesita, con **flutter pub add**, y sus versiones se verifican en pub.dev (mantenimiento y compatibilidad con tu versión de Flutter) antes de fijarlas.

3. Primer avance: los mockups en VS Code

Objetivo. Construir en Flutter, con datos de ejemplo, el flujo principal de los mockups y mostrarlo corriendo en un teléfono Android real. No se necesita backend terminado ni lógica real: lo importante es la fidelidad visual, la navegación completa y el cambio de idioma funcionando. La fecha del avance está por confirmar; el plan se da para **5 días** y trae una variante de **3 días**.

3.1 Pantallas del avance y prioridad

A = indispensable · B = deseable · C = solo si sobra tiempo · — = no entra en el avance.

#	Pantalla del mockup	Prior.	Qué construir y qué cambiar respecto al mockup
1	Entrada (Google / correo)	A	Solo interfaz. Ambos botones llevan al onboarding. Enlaces de términos y privacidad como texto.
2	Onboarding: perfil	A	Dos tarjetas seleccionables (Visitante o Instalación). El perfil se guarda en estado local con Riverpod. Saludo con nombre de ejemplo.
3	Mapa de departamentos con buscador	A	Formas provisionales con la interacción final: tocar “Cesar” abre el selector de municipios. **Quitar el micrófono** del buscador (voz es Fase 2). El mapa real con geometría oficial es semana 2.
4	Selector de municipios	A	Autocompletado sobre una lista local de municipios de Cesar, sin distinguir mayúsculas ni tildes. Valledupar marcado como piloto.
5	Municipio piloto (Valledupar)	A	Imagen de ciudad (con licencia o atribución), insignia “Disponible en el piloto” y botón Entrar.
6	Municipio fuera del piloto	A	Mensaje honesto de “información no verificada”. El botón “Notificarme” solo registra el interés (sin notificaciones) y pasa a “Te avisaremos”.
7	Inicio, perfil Instalación	A	Tarjeta “¿Te están cobrando lo justo?” y herramientas. Trámites y Documentos aparecen con la etiqueta “Próximamente” si no entran; Conversor y Directorio se ocultan (Fase 2).
8	Inicio, perfil Visitante	A	Igual, más el banner del evento. El banner debe usar un color distinto al ámbar del semáforo.
9	Perfil y ajustes	A	Cambio de idioma real español/inglés, cambio de perfil, ciudad actual y cerrar sesión (vuelve a la entrada).
10	Analizar (precio o contrato)	B	Selector de tipo y botones de archivo/foto sin lógica real; al confirmar muestra un estado de carga de unos segundos y abre el resultado de ejemplo.
11	Resultado del análisis	B	Semáforo ámbar con porcentaje, referencia y contexto del evento, y pestaña de contrato con puntos clave de ejemplo. Cambiar “promedio” por “precio de referencia”. Incluir el aviso de que es orientativo.
12	Aportar un precio (4 pasos)	B	Flujo con validación local (monto mayor que 0, categoría elegida) y confirmación. No guarda en ningún servidor.
13	Mapa de precios	C	Con Google Maps si ya hay clave y facturación; si no, imagen estática con marcadores dibujados. Chips Precios y Servicios, y botón “Aportar un precio”.
14	Detalle de servicio / negocio	C	Reutilizar el diseño del hospital para un negocio de ejemplo con botones Pedir, WhatsApp, Llamar y Cómo llegar.
15	Mis trámites y Mis documentos	C	Solo si sobra tiempo; son “si hay tiempo” en los requisitos.
16	Dinero (conversor y gastos)	—	Fase 2. No se construye ni se muestra.

#	Pantalla del mockup	Prior.	Qué construir y qué cambiar respecto al mockup
17	Vista de ciudad con cultura; registro de negocio	—	No tienen mockup: diseñarlos en Figma primero. Se construyen en la semana 5.

Decisión que conviene tomar el primer día: la barra inferior definitiva. Los mockups cambian de pestañas según el perfil (Guardados o Dinero) y eso complica la navegación. Propuesta: cinco pestañas fijas — Inicio, Mapa, Escanear (central), Negocios y Perfil — y mostrar “Próximamente” en lo que no entre.

3.2 Preparar el entorno en VS Code

1. Instalar el SDK de Flutter y Android Studio (SDK de Android y herramientas de línea de comandos) siguiendo la guía oficial de flutter.dev, y aceptar licencias con `flutter doctor --android-licenses`.
2. En VS Code, instalar las extensiones **Flutter** y **Dart**.
3. Conectar un teléfono Android con depuración USB (o crear un emulador) y comprobar con `flutter devices`. **Prueba en teléfono real** desde el primer día.
4. Ejecutar `flutter doctor` hasta que no tenga errores. Esto es lo que más tiempo puede quitar: hazlo antes que cualquier otra cosa.
5. Crear el repositorio en GitHub y, dentro de él, el proyecto: `flutter create --platforms=android --org app`. **El nombre de paquete (applicationId) es difícil de cambiar después** y las claves de Google se restringen por ese nombre: defínelo antes (pendiente).
6. Agregar dependencias del avance: `flutter pub add flutter_riverpod go_router easy_localization flutter_svg`. Comprobar versiones en pub.dev.
7. Configurar en pubspec.yaml los *assets*: carpeta de traducciones, fuente Inter y SVG. Copiar app_theme.dart a lib/core/theme/ y usarlo en MaterialApp.
8. Flujo diario: `flutter run` y recarga en caliente (tecla r). Una rama por pantalla: `git checkout -b feature/onboarding`.

3.3 Orden de construcción y cronograma

Día	Frontend (tú)	Backend (compañero, en paralelo)	Entregable del día
1	Entorno, proyecto, repo, tema, easy_localization, go_router con rutas vacías y 4 componentes compartidos (botón, tarjeta, insignia, campo de búsqueda). Pantallas 1 y 2.	Confirmar D4. Proyecto Supabase, estructura supabase/ en el repo, Supabase CLI y entorno local.	App con tema y es/en corriendo en el teléfono
2	Pantallas 3, 4, 5 y 6 (mapa provisional, selector, piloto y no piloto) con la lista local de municipios.	Migración 0001 con el esquema v0 (sección 4.3) y semilla de departamentos y municipios de Cesar con Valledupar como piloto.	Flujo entrada → municipio completo
3	Pantallas 7, 8 y 9 (inicios, barra inferior fija, perfil con idioma funcionando).	Autenticación configurada (Google y correo). Documento del contrato de datos v0.1. Diagrama ER en PlantUML.	Navegación completa en es/en
4	Pantallas 10, 11 y 12 con datos de ejemplo (fixtures): análisis, resultado de semáforo y aportar precio.	Función SQL del semáforo con sus pruebas (puede ir sin conectar a la app todavía).	Núcleo del producto visible
5	Pulido: texto grande y 360 dp sin desbordes, versión en inglés revisada, estados vacíos y de error básicos. Pantallas 13 a 15 si hay tiempo. Prototipo de Figma del flujo. Ensayo y etiqueta v0.1-avance.	Revisión del repositorio: sin secretos, README para ejecutar, issues cargados por semana.	Demo ensayada y repositorio limpio

Variante de 3 días. Día 1: entorno, tema, idiomas, router, pantallas 1 y 2. Día 2: pantallas 3 a 6 y 7/8. Día 3: perfil con idioma, resultado del semáforo (pantalla 11 con datos de ejemplo) y pulido. Se omiten las pantallas 10, 12 y siguientes, y el prototipo de Figma cubre el flujo que no alcanzó a programarse.

3.4 Método para construir cada pantalla

- **Medir en Figma** (panel Inspect): espaciados, radios, tamaños de texto y sombras. No copies el código que genera Figma; construye con tus componentes compartidos.
- **Estructura**: cada pantalla vive en lib/features/*módulo*/presentation/. Los datos llegan por un provider que lee un repositorio; en el avance el repositorio devuelve datos falsos (sección 5.2).
- **Textos**: todo con `'clave'.tr()`, claves del tipo `home.scan.title`. Agrega siempre la clave en es.json y en en.json.
- **Colores**: solo desde el tema. Verde, ámbar y rojo exclusivamente para el semáforo; el aviso del evento con otro color.
- **Checklist por pantalla** antes de abrir el PR: se ve como el mockup; sin desbordes a 360 dp y con texto grande del sistema; botón atrás correcto; áreas táctiles de al menos 48 dp; contraste mínimo 4,5:1; textos en español e inglés; sin texto en duro; `flutter analyze` sin errores; capturas en el PR.

3.5 Datos de ejemplo que debes preparar

- Lista de municipios de Cesar con código DIVIPOLA (Valledupar es el piloto) en un JSON local; verifica los códigos contra el dataset oficial.
- Un resultado de semáforo ámbar (por ejemplo, precio de 15.000 COP frente a una referencia de 12.000, 25 % por encima) y uno verde y uno rojo para probar los tres estados.
- Un contrato de ejemplo con dos puntos clave, en español y en inglés.
- Un evento de demostración, **marcado como demo en pantalla**, para mostrar el banner. El Festival Vallenato no estará en curso en noviembre; no presentes datos simulados como reales.

3.6 Qué hace el backend para el avance

No bloquea la interfaz. Lo mínimo que debe quedar en el repositorio: proyecto Supabase creado, migración inicial con el esquema v0, semilla de departamentos y municipios de Cesar, autenticación configurada, contrato de datos v0.1 (tablas y campos) y el diagrama ER. Si D4 no está confirmada a tiempo, el equipo presenta el esquema SQL y el ER como diseño propuesto, y la app sigue con datos de ejemplo.

3.7 Guion de presentación (unos 7 minutos)

Minuto	Qué se muestra	Notas
0–1	El problema con evidencia: en el Festival Vallenato, la tarifa mínima oficial de taxi era de 8.300 pesos y se reportaron cobros de 70.000 a 100.000.	Citar las fuentes del documento v3.
1–2	La propuesta: semáforo de justicia, mapa con piloto honesto, dos perfiles.	Explicar que el semáforo de precio es una regla verificable, no una caja negra.
2–4,5	Demo en el teléfono: entrada, onboarding, mapa, Cesar, Valledupar, inicio, escanear, resultado ámbar con contexto, cambio a inglés.	Decir en voz alta qué es interfaz con datos de ejemplo.
4,5–5,5	Arquitectura y decisiones: Flutter, Supabase, Gemini y Google Maps, y por qué.	Mostrar el diagrama de la sección 4.
5,5–6,5	Plan de seis semanas, riesgos y lo que falta por definir.	Mostrar la tabla de pendientes.
6,5–7	Qué se pide al profesor: opinión sobre el alcance del MVP y sobre la tarifa oficial como tope.	

Rotula todo con honestidad: “Implementado (interfaz con datos de ejemplo)”, “Prototipo de Figma” y “Planificado”. Un jurado valora más un alcance declarado con claridad que una promesa exagerada, y es coherente con el principio del producto de no simular datos.

3.8 Criterios de aceptación del avance

- La app se instala y recorre el flujo de la demo en un **teléfono Android real**, sin errores.
- Las pantallas A cumplen el checklist de la sección 3.4; las B están hechas o marcadas como pendientes en el tablero.
- El cambio español/inglés funciona en todo el flujo, y un test automático verifica que es.json y en.json tienen las mismas claves.
- Navegación sin callejones sin salida; el botón atrás se comporta bien.
- Repositorio en GitHub con main protegida, un PR por pantalla o grupo de pantallas, README con cómo ejecutar, y sin claves ni secretos en el historial.
- Documento de requisitos y este plan entregados; esquema SQL v0 y ER si D4 está confirmada.

3.9 Riesgos del avance y plan B

Riesgo	Plan B
La instalación de Flutter o del SDK de Android falla o se demora	Hacerlo primero. Si el teléfono no conecta, usar emulador; pedir ayuda al compañero ese mismo día.
No hay clave ni facturación de Google a tiempo	Mapa de precios con imagen estática y marcadores dibujados; se rotula como “mapa de ejemplo”.
Riverpod o go_router se atascan	Usar la versión mínima: un provider simple por pantalla y rutas planas; refinar en la semana 2.
No alcanza el tiempo	Recortar por prioridad: A primero, luego B, y C solo si sobra. El prototipo de Figma cubre lo demás.
Desbordes de texto en inglés o en pantallas pequeñas	Probar desde el día 2 a 360 dp y con texto grande; usar Flexible/Expanded y evitar alturas fijas.
Imágenes sin licencia clara	Usar fotos propias o con licencia que permita su uso, con atribución; si no, un recuadro neutro.

4. Arquitectura, estructura y modelo de datos

Es una arquitectura **cliente + backend como servicio**, sin servidor propio ni microservicios: es la opción más simple que cubre los requisitos con dos personas y seis semanas. La app nunca habla con Gemini; lo hace una Edge Function, de modo que la clave no está en el teléfono.

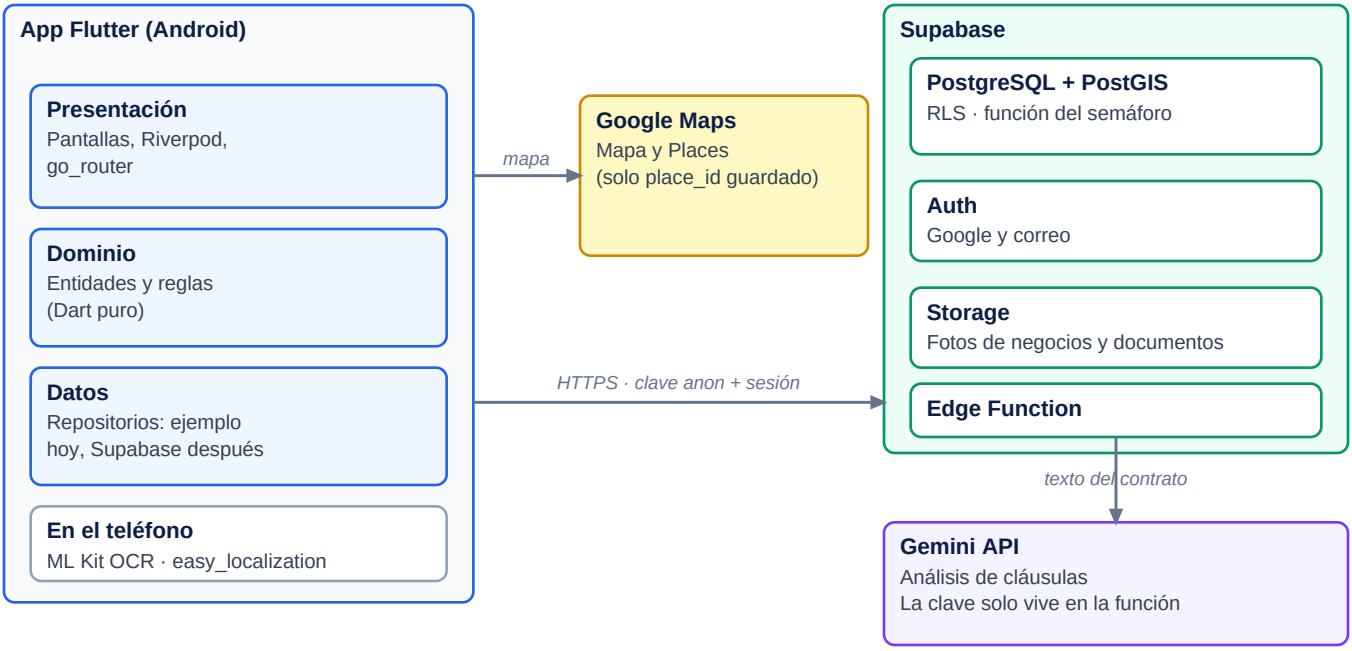


Figura 1. Componentes y comunicación. Estado: D4 propuesta (falta confirmar Supabase).

4.1 Capas de la app y reglas de dependencia

Capa	Contiene	Regla
Presentación	Pantallas, widgets, providers de Riverpod	No importa la capa de datos. Solo conoce entidades y repositorios abstractos.
Dominio	Entidades (Report, Business, Verdict) e interfaces de repositorio	Dart puro, sin Flutter ni Supabase. Es lo más fácil de probar.
Datos	Repositorios concretos (de ejemplo y de Supabase) y conversión de JSON	Lo único que cambia al pasar de datos de ejemplo a reales. Se elige con un provider.

4.2 Estructura del repositorio y de la app

```
via-justa/
  app/                      Flutter (solo Android)
    lib/
      core/                 tema, rutas, idiomas, utilidades, componentes compartidos
      features/
        onboarding/ map_colombia/ prices/ contracts/ city/ businesses/ profile/
          presentation/  domain/  data/
      main.dart
    assets/translations/    es.json  en.json
    test/
  supabase/
    migrations/             SQL versionado (nunca cambios manuales en el panel)
    functions/analyze_contract/
    seed/                   departamentos, municipios, categorias, datos semilla
  docs/                     requisitos, contrato-datos, decisiones, uml/
  .github/                  plantillas de issue y PR, CI
  CLAUDE.md                 reglas para el agente de código
  README.md
```

4.3 Esquema de datos v0 (borrador para el contrato de datos)

Es la base para la migración 0001 y para el diagrama ER. Los nombres están en inglés y en snake_case [Propuesto]; el ER definitivo se cierra con el compañero de backend.

Tabla	Campos principales	Notas
profiles	id (= usuario de Auth), display_name, profile_type (visitor | settler), origin_country, current_municipality_code, language, role	role no debe poder editarlo el usuario (RLS o claims).
consents	user_id, doc_type, version, accepted_at	Registro del consentimiento (Ley 1581).
departments / municipalities	code (DIVIPOLA), name, department_code, is_pilot	Carga única desde DANE. is_pilot decide qué municipios abren la app funcional.
price_categories	id, key, sort_order	Lista pendiente de definir (por ejemplo transporte, alimentación, alojamiento).
price_reports	id, user_id, category_id, municipality_code, amount_cop, location (geography), is_seed, is_hidden, created_at	Índice espacial y por (municipio, categoría, fecha).
events	id, municipality_code, name, starts_at, ends_at, is_demo	Las crea el administrador.
app_config	key, value (jsonb)	Umbrales 15 y 40, N = 3, ventana 60 días, límites por día, factor 1,15.
ai_usage	user_id, day, count	Tope de análisis por usuario y por día.
businesses	id, owner_id, type, name, description_es, description_en, location, place_id, municipality_code, phone, whatsapp, whatsapp_consent_at, order_url, hours (jsonb), status, rejection_reason	status: pending | approved | rejected.
business_photos / business_dishes	business_id, path, position / name, price_cop, description	Máximo 5 fotos y 5 platos destacados.
city_content	municipality_code, locale, title, body, image_path	Cultura local en es y en.
interest_registrations	user_id, municipality_code	Para el botón “Notificarme” (solo se registra).

4.4 Convenciones [Propuesto]

- **Dart**: archivos en snake_case.dart, clases en UpperCamelCase, providers terminados en Provider, identificadores en inglés.
- **SQL**: tablas en plural y snake_case, claves primarias id (uuid), fechas con zona horaria, montos en COP como entero.
- **Traducciones**: claves del tipo modulo.pantalla.elemento; datos fijos como claves (por ejemplo la categoría transport), no como texto.
- **Git**: ramas feature/…, fix/…, docs/…; commits tipo feat(prices): … en español; PR con lista de verificación y capturas.
- **Documentación**: cada decisión nueva se anota en docs/decisiones.md con fecha, motivo y alternativa descartada.

5. Guía de implementación por módulo

Cada módulo indica qué hace el frontend y qué hace el backend. Lo marcado [Pendiente] depende de decisiones abiertas de la sección 10.

5.1 Idiomas con easy_localization (D1, D2)

- Crear assets/translations/es.json y en.json con las **mismas claves**; declarar la carpeta en pubspec.yaml.
- Inicializar el paquete en main.dart con locales es y en, idioma de respaldo es y la ruta de traducciones; envolver la app con su widget y pasar los delegados a MaterialApp (seguir la guía del paquete).

- Usar `.tr()` en cada texto. Detectar el idioma del teléfono al inicio y permitir cambiarlo en Perfil sin reiniciar; el paquete guarda la elección.
- Test automático: lee ambos JSON y falla si una clave existe en uno y no en el otro.
- Backend: los datos curados (cultura, checklist) llevan columnas o filas por idioma; los datos fijos se guardan como clave. Los errores del servidor son códigos que la app traduce.

5.2 Navegación y estado: Riverpod y go_router (D5)

- Declarar todas las rutas en core/router.dart: `/login`, `/onboarding`, `/map`, `/municipio/:code`, `/home`, `/prices/verify`, `/prices/result`, `/contracts/analyze`, `/businesses/:id`, `/profile`.
- Usar la redirección de go_router para el flujo de entrada: sin sesión → login; sin perfil → onboarding; sin municipio → mapa.
- **Patrón de datos**: una interfaz de repositorio en dominio (por ejemplo, un repositorio de verificación de precios), una implementación de ejemplo y otra de Supabase. Un provider devuelve la que corresponda; al conectar el backend se cambia solo ese provider, sin tocar pantallas.
- Estados de pantalla con AsyncValue: cargando, datos y error, siempre con mensaje traducido y reintento.

5.3 Semáforo de precio (regla estadística, no IA)

Se calcula en el servidor, en una sola función, para que haya una única fuente de verdad. La app solo muestra el resultado.

```
verdict(categoria, municipio, precio):
  evento   = evento activo del municipio (hoy entre starts_at y ends_at)
  reportes = reportes visibles (no semilla ocultos), misma categoria y municipio,
             de los ultimos 60 dias        -- ventana tomada de app_config
  n, ref   = count(*), percentile_cont(0.5) sobre amount_cop   -- mediana
  si n < 3 -> { estado: 'insuficiente' }                       -- sin color
  desv     = (precio - ref) / ref * 100
  nivel    = desv <= 15 ? verde : desv <= 40 ? amarillo : rojo
  aviso_bajo = desv < -40                    -- verde con aviso informativo
  si hay evento y hay >= 3 reportes dentro de sus fechas:
      ref_evento = mediana de esos reportes; calcular tambien desv_evento
      el nivel mostrado usa ref_evento; el resultado trae ambas comparaciones
  tope_recomendado = referencia vigente * 1.15
  devolver { nivel, desv, ref, n, aviso_bajo, evento: {...}, tope_recomendado }
```

- **Pruebas obligatorias** (RNF-09): 0 %, 15 %, 15,01 %, 40 %, 40,01 %, menos de 3 reportes, precio 40 % por debajo, evento con y sin reportes suficientes.
- **Frontend**: un componente de tarjeta de semáforo que lee los colores de la extensión del tema; el nivel siempre lleva color, ícono y texto.
- **Gancho para el futuro**: dejar una columna opcional de tarifa oficial por categoría y municipio, para P6, sin implementarla aún.

5.4 Mapa de departamentos y municipios

1. Obtener la geometría oficial de departamentos (Marco Geoestadístico Nacional del DANE) y verificar sus términos de uso [Pendiente].
2. Simplificar la geometría con una herramienta como mapshaper hasta un archivo liviano, y exportar un SVG con un id por departamento igual al código DANE (por ejemplo, 20 para Cesar).
3. En Flutter, convertir cada trazo a un Path (por ejemplo con un paquete de interpretación de rutas SVG, verificando su mantenimiento), dibujarlo con CustomPainter y detectar el toque con Path.contains. Envolver en InteractiveViewer para el zoom.
4. Pintar el departamento seleccionado y mostrar su escudo en SVG; para los que no tengan símbolo, uno genérico. El tratamiento de Bogotá D.C. está pendiente (P9).
5. Buscador de municipios: tabla municipalities con búsqueda sin tildes (extensión unaccent de PostgreSQL) y autocompletado con una espera de 300 ms entre pulsaciones.

En el avance se usan formas provisionales con la misma interacción; el mapa real es de la semana 2.

5.5 Mapa de precios con Google Maps (D3)

- **Antes de escribir código**: crear la cuenta de facturación de Google Cloud, activar las APIs necesarias, fijar alertas de presupuesto y cuotas diarias, y crear la clave de Android restringida por nombre de paquete y huella SHA-1. La clave va en un archivo ignorado por Git.
- Consultar reportes solo del área visible del mapa y con un máximo (por ejemplo, 200); agrupar marcadores o limitar los más cercanos si hay muchos.
- Cada marcador usa el color y el ícono del semáforo; filtro por categoría.
- Places para ubicar negocios: guardar solo el place_id y las coordenadas del propio usuario; el resto se consulta cuando se necesita. Verificar costos y términos vigentes.

5.6 Análisis de contratos con Gemini (D9)

1. **Captura**: foto o archivo (JPG, PNG o PDF hasta 10 MB). Para imágenes, reconocimiento de texto en el teléfono; para PDF, extraer el texto nativo. El PDF escaneado está pendiente (P10).
2. **Llamada**: la app envía solo el texto y el idioma a la Edge Function, con la sesión del usuario.
3. **Función**: valida la sesión, comprueba el tope diario (ai_usage), limita el largo del texto, arma el prompt y llama a Gemini pidiendo una salida en JSON con esquema fijo (verificar en la documentación de la API cómo se declara). Valida el JSON recibido y, si es inválido, devuelve un error controlado.
4. **Seguridad del prompt**: el texto del contrato es dato no confiable. El prompt indica que se ignoren instrucciones que aparezcan dentro del contrato, y la respuesta se valida contra el esquema antes de mostrarla.
5. **Privacidad**: el texto no se guarda (RN-10); solo se registra el conteo de uso. Con el nivel gratuito, usar únicamente contratos de prueba.
6. **Resultado**: puntos clave, nivel, cláusulas señaladas con referencia normativa cuando aplique y recomendación, en el idioma del usuario y con el aviso de que es orientativo. Las referencias legales de los prompts se validan contra el texto oficial.

```
{ "level": "green|yellow|red",
  "key_points": [ { "title": "...", "detail": "..." } ],        // 2 o 3
  "clauses": [ { "excerpt": "...", "issue": "...",
                 "legal_reference": "Ley 820 de 2003, art. ... | null",
                 "recommendation": "..." } ],
  "general_recommendation": "...",
  "language": "es|en" }
```

- **Errores a manejar**: sin conexión, texto vacío o ilegible, tope diario alcanzado (429), respuesta inválida del modelo y tiempo agotado. Cada uno con mensaje traducido.
- **Frontend**: estados de progreso (leyendo, analizando, listo) y pantalla de resultado con pestañas Precio y Contrato.

5.7 Negocios y servicios locales (D6, D7)

- **Formulario** de 4 pasos con los campos de la sección de requisitos: tipo, nombre, descripción, ubicación, teléfono, WhatsApp con consentimiento, horarios, fotos y, según el tipo, carta o enlaces.
- **Fotos**: bucket de Storage con límite de tamaño y de cantidad; reducir la imagen antes de subirla.
- **RLS**: cualquiera lee negocios aprobados; el dueño crea y edita los suyos; solo el administrador cambia el estado. Todo negocio nuevo nace como pending.
- **Moderación en el MVP**: el administrador aprueba o rechaza desde el panel de Supabase; una pantalla de administración solo si sobra tiempo.
- **Contacto**: WhatsApp con un enlace wa.me (número en formato internacional sin el signo más) y mensaje prellenado según el idioma; Llamar; Cómo llegar (abre Google Maps); Pedir solo si hay enlace. Usar url_launcher y declarar en Android las consultas que su documentación pide para abrir enlaces.
- **Alcance**: un negocio solo se registra en un municipio piloto [Propuesto].

5.8 Eventos y datos semilla

- **Eventos**: el administrador crea eventos por municipio con fechas; el inicio muestra el banner mientras estén activos. Para la demo, un evento marcado como demostración.
- **Datos semilla**: precios **reales** observados en Valledupar (nunca inventados), marcados como is_seed y con su fuente. Para superar el mínimo de 3 reportes, al menos 5 por categoría. Empezar a recolectarlos en la semana 2: es trabajo de campo que no se puede dejar para el final.

5.9 Seguridad y privacidad transversales

- RLS activa en todas las tablas; ningún secreto (Google, Gemini, service_role) en la app ni en el repositorio. La clave anon de Supabase se lee de un archivo de entorno ignorado por Git.
- Si una clave se sube por error: revocarla y generar otra; borrar el commit no basta.
- Consentimiento registrado con fecha y versión antes de usar la app; política de tratamiento de datos publicada; eliminación de cuenta y de sus datos (RF-06).
- Documentos personales (si entran) en almacenamiento privado, con acceso solo del dueño. No afirmar “cifrado” en pantalla si no está configurado.

6. Plan por semanas

Las fechas son las de la semana calendario. Los festivos probables del 2 y el 16 de noviembre deben verificarse en el calendario oficial. La **congelación de funcionalidades** es al final de la semana 5 (22 de noviembre); la semana 6 es para pruebas, correcciones y ensayo. El esfuerzo es relativo: S (horas), M (1 a 2 días), L (varios días). Las horas disponibles por persona están por definir; si son pocas, se recorta alcance, no calidad.

Semana 1 · 19–25 de octubre · PRIMER AVANCE

Objetivo	Mockups en Flutter navegables con datos de ejemplo, y bases del backend y del repositorio.
Frontend (tú)	Sección 3: entorno, tema, idiomas, router, componentes y pantallas 1 a 9 (A), luego 10 a 12 (B) (M a L).
Backend (compañero)	Confirmar D4, proyecto Supabase, migración 0001, semilla de Cesar, autenticación configurada, contrato de datos v0.1 y ER (M).
Entregable	App en el teléfono, repositorio con estructura y CI, documentos, esquema SQL, prototipo de Figma.
Terminado cuando	Se cumplen los criterios de aceptación de la sección 3.8.

Semana 2 · 26 de octubre–1 de noviembre · Cuenta y mapa

Objetivo	Pasar de datos de ejemplo a datos reales en el flujo de entrada.
Frontend (tú)	Conectar supabase_flutter; autenticación con Google y correo; guardar perfil, idioma y consentimiento; mapa real de departamentos con SVG; buscador de municipios contra la base; vista de municipio piloto o no piloto desde is_pilot (L). Diseñar en Figma la vista de ciudad y el registro de negocio (M).
Backend (compañero)	RLS de profiles, consents y municipios; carga completa de DIVIPOLA; función de eliminación de cuenta; tabla de interés (M). Empezar la recolección de datos semilla en Valledupar.
Entregable	Flujo real: iniciar sesión → perfil → mapa → municipio, con la sesión persistente.
Terminado cuando	Un usuario nuevo completa el flujo y sus datos quedan guardados; un usuario sin permisos no puede leer lo ajeno.

Semana 3 · 2–8 de noviembre · Precios y semáforo

Objetivo	El núcleo estadístico funcionando de punta a punta.
Frontend (tú)	Aportar un precio (4 pasos) con ubicación; mapa de Google con marcadores y filtro; pantalla de verificación y resultado conectada a la función; banner de evento (L).

Backend (compañero)	Tablas de categorías, reportes, eventos y configuración; RLS y límite diario de reportes; función del semáforo con sus pruebas; carga de semillas; el evento de demostración (L).
Entregable	Flujo de precios completo en el teléfono, con semillas de Valledupar.
Terminado cuando	Las pruebas de límites del semáforo pasan y el resultado coincide con el mockup en los tres niveles.

Semana 4 · 9–15 de noviembre · Contratos con IA

Objetivo	Análisis de contratos real, seguro y con costo controlado.
Frontend (tú)	Captura por cámara o archivo; reconocimiento de texto; pantalla de progreso y resultado con pestañas; manejo de errores (L).
Backend (compañero)	Edge Function con Gemini, esquema de salida, validación, tope diario y secretos; prompts con referencias legales validadas; pruebas con 3 a 5 contratos de ejemplo (L).
Entregable	Flujo de contrato de punta a punta, en español e inglés.
Terminado cuando	Con contratos de prueba, el resultado es válido, se muestra en el idioma elegido y los errores se manejan sin cerrar la app.

Semana 5 · 16–22 de noviembre · Ciudad y negocios · congelación

Objetivo	Lo pedido por el profesor: cultura local y negocios de los usuarios.
Frontend (tú)	Vista de ciudad con contenido es/en; formulario de negocio en 4 pasos; detalle con carta, WhatsApp, Llamar, Cómo llegar y Pedir; listado y mapa de negocios (L).
Backend (compañero)	Tablas y Storage de negocios, RLS y estados de moderación; contenido cultural de Valledupar; flujo de aprobación (L).
Entregable	Negocio registrado → aprobado → visible y contactable.
Terminado cuando	Un negocio pasa por todo el ciclo y un usuario ajeno no puede editarlo. Se congelan las funcionalidades.

Semana 6 · 23–29 de noviembre · Cierre

Objetivo	Calidad, seguridad, documentación y ensayo.
Frontend (tú)	Pruebas en teléfono real, revisión de traducciones y desbordes, accesibilidad, APK de lanzamiento, ensayo de la sustentación (M).
Backend (compañero)	Revisión de RLS y secretos, alertas de presupuesto, datos semilla finales, respaldo de la base, README y manual (M).
Entregable	APK instalable, documentación y presentación.
Terminado cuando	Se cumple la definición de terminado (sección 8) y el repositorio queda limpio. La fecha exacta de entrega está por confirmar.

> **Si todo va bien:** la bolsa “si hay tiempo” (checklist de trámites y carpeta de documentos) puede ocupar la semana 5 en paralelo. **Si algo se atrasa**, se recorta en este orden: versión completa de negocios (queda el registro básico), símbolos de los departamentos, historial de análisis.

7. Diseñado para el cambio

Los requisitos van a cambiar (el feedback del profesor ya cambió el alcance una vez). Estas decisiones hacen que los cambios sean baratos:

Qué puede cambiar	Cómo lo absorbe el diseño
Umbrales del semáforo, N, ventana de días, límites por día, factor de recomendación	Viven en la tabla app_config, no en el código. Se cambian sin publicar una versión nueva.
Proveedor o modelo del LLM	Está aislado en una sola función (analyze_contract). La app no sabe qué modelo hay detrás.
Idiomas nuevos	Agregar un archivo de traducciones y una opción en el selector. Los datos curados ya tienen una fila por idioma.
Nuevas ciudades	Es un cambio de datos: marcar is_pilot y cargar semillas y contenido cultural. No toca código.
Nuevos tipos de negocio o categorías de precio	Se agregan como valores de lista o filas de tabla; el formulario muestra campos según el tipo.
Tarifa oficial como tope (P6)	Columna opcional prevista en el diseño; se activa cuando se decida.
Módulos de Fase 2 (directorio, conversor, foro, reseñas)	Cada uno entra como una carpeta nueva en features/, sin tocar las existentes. Los que no están listos se ocultan con una constante de configuración.
iOS	El proyecto Flutter puede agregar la plataforma luego (flutter create --platforms=ios en la misma carpeta); implica claves, configuración y pruebas propias.
Cambios en la base de datos	Toda modificación es una migración numerada; el contrato de datos se versiona en docs/ y cambia solo con acuerdo de ambos.
Cambios de diseño	Se hacen primero en Figma (estilos y componentes) y luego en app_theme.dart; los colores y radios están centralizados.

Cómo gestionar un cambio de requisito

1. Anotarlo en docs/decisiones.md: qué cambia, por qué y qué alternativa se descartó.
2. Actualizar el documento de requisitos (ID afectados) y, si toca datos, el contrato de datos.
3. Crear o ajustar los issues con su semana; si algo entra, decidir qué sale (el alcance no crece sin recortar).
4. Hacer el cambio en una rama, con pruebas y revisión del otro integrante.

8. Calidad, seguridad y definición de terminado

Nivel	Qué se prueba	Cómo
Unitarias	Regla del semáforo (en la función SQL y en el modelo de la app), validaciones de formularios, conversión de JSON, claves de traducción	flutter_test y pruebas SQL con casos límite
Widgets	Pantallas clave en es y en, estados cargando, vacío y error	flutter_test con repositorios de ejemplo
Integración manual	Flujos completos en un teléfono Android real: entrada, precios, contratos, negocios	Lista de verificación por flujo, con capturas
Seguridad	RLS (un usuario no lee ni edita lo ajeno), secretos fuera del repositorio, tope de análisis	Pruebas con dos cuentas y revisión del historial de Git
Contratos de IA	Respuesta válida, idioma correcto, cláusulas detectadas, error controlado	3 a 5 contratos de prueba (con datos personales borrados)

Nivel	Qué se prueba	Cómo
Accesibilidad	Contraste 4,5:1, texto mínimo 12 sp, semáforo no solo por color	Verificador de contraste y prueba con texto grande

Definición de terminado de una tarea

- Cumple los criterios de aceptación de su historia y no rompe los requisitos relacionados.
- Tiene pruebas donde corresponde, `flutter analyze` sin errores, y fue probada en un teléfono real si toca cámara, mapa, ubicación o contacto.
- Textos en español e inglés, sin texto en duro; sin secretos ni datos personales en el código.
- Documentación actualizada (contrato de datos o decisiones) y PR revisado por el otro integrante.
- Si se usó un agente de código: se revisó todo el cambio, y se anotó en el PR o en la documentación según lo pida el curso.

9. Riesgos del proyecto

Riesgo	Prob.	Impacto	Mitigación
D4 sin confirmar o Supabase no cumple	Media	Alto	Confirmar el día 1; repositorios de ejemplo permiten avanzar sin backend; el esquema SQL es portable.
Alcance mayor que el tiempo	Alta	Alto	Prioridades A/B/C, congelación en la semana 5 y orden de recorte definido.
Curva de aprendizaje de Flutter, Riverpod y go_router	Alta	Medio	Ejemplos pequeños antes de usarlos; versión mínima de cada uno; revisión de código cruzada.
Datos semilla insuficientes: el semáforo dice “datos insuficientes”	Media	Alto	Recolectar desde la semana 2, al menos 5 reportes reales por categoría.
Calidad del análisis de contratos en español jurídico	Media	Medio	Probar con contratos reales de prueba, ajustar el prompt, validar las referencias legales.
Costos y privacidad del LLM y de Google Maps	Media	Medio	Tope por usuario, alertas de presupuesto, nivel gratuito solo con contratos de prueba.
Mapa SVG de departamentos más difícil de lo previsto	Media	Medio	Versión sencilla primero (formas tocables); símbolos como mejora.
Spam o datos falsos en reportes y negocios	Media	Medio	Mediana, límites diarios, moderación de negocios y ocultar reportes.
El feedback del profesor cambia el alcance otra vez	Media	Medio	Sección 7: gestión de cambios y diseño modular.
Dependencia entre los dos integrantes	Media	Alto	Contrato de datos acordado primero y repositorios de ejemplo en el frontend.
Problemas de instalación o de dispositivo	Media	Medio	Resolver el entorno el día 1; tener teléfono real y emulador.

10. Pendientes por definir

Cada pendiente trae una recomendación por defecto y el momento límite para decidirla. Si no se decide a tiempo, se aplica la recomendación y se anota en el registro de decisiones.

#	Pendiente	Quién	Recomendación por defecto	Límite
1	Fecha y formato del primer avance y de la entrega final	Profesor / equipo	Plan de 5 días; entrega con funcionalidades congeladas el 22 de nov.	Hoy
2	D4: confirmar Supabase	Compañero de backend	Supabase	Día 1 del avance
3	D8: arquitectura por capas y agente de código con archivo de reglas	Tú	Aprobar	Día 1
4	Horas disponibles por persona	Ambos	Ajustar alcance con base en ellas	Hoy
5	Política del curso sobre uso de IA y cómo declararlo	Tú con el profesor	Declarar en el README y los PR	Antes del avance
6	Nombre de paquete Android (applicationId)	Ambos	Dominio invertido corto y definitivo	Día 1
7	Barra inferior definitiva	Tú	Inicio, Mapa, Escanear, Negocios, Perfil	Día 1
8	Convención de idioma del código	Ambos	Código en inglés, documentación en español	Día 1
9	Cuenta de facturación de Google Cloud y clave de Maps	Ambos	Crear una con alertas de presupuesto	Antes de la semana 3 (antes del avance si se quiere mapa real)
10	P6: tarifa oficial regulada como tope	Ambos	Sí: mostrarla como referencia y no superarla	Semana 2
11	P8: segunda ciudad del piloto	Ambos	Solo Valledupar	Semana 2
12	P9: Bogotá D.C. en el mapa	Tú	Región adicional junto a los 32 departamentos	Semana 2
13	P10: PDF escaneado	Ambos	Solo fotos y PDF con texto	Semana 3
14	P11: onboarding por perfil o por tiempo de estadía	Ambos	Por perfil, como el mockup	Antes de la semana 2
15	Lista de categorías de precio del MVP	Ambos	Transporte, alimentación, alojamiento y otros	Semana 2
16	Fuente y licencia de la geometría de departamentos	Tú	Marco Geoestadístico del DANE si sus términos lo permiten	Semana 2
17	Símbolos de departamentos e imágenes de ciudad: fuentes y licencias	Tú	Pocos símbolos y uno genérico; imágenes con licencia y atribución	Semana 2
18	Versión mínima de Android	Tú	La que exijan Flutter, Google Maps y ML Kit	Día 1
19	Modelo exacto de Gemini y política de datos aplicable	Compañero	Un modelo de la familia Flash; contratos de prueba en el nivel gratuito	Semana 4
20	Contenido cultural de Valledupar (qué y de qué fuentes)	Ambos	Breve, en es y en, con fuentes citadas	Semana 4
21	Quién modera los negocios y con qué criterios	Ambos	El administrador, con motivo de rechazo	Semana 5
22	Alcance del panel de administración	Ambos	Panel de Supabase en el MVP	Semana 5

Qué hacer hoy

1. Hablar con tu compañero: confirmar D4 y repartir la semana 1; compartirle este documento y el de requisitos.
2. Resolver los pendientes del día 1 (filas 2, 3, 6, 7, 8 y 18) y confirmar la fecha del avance con el profesor.
3. Instalar y verificar el entorno (sección 3.2). Es lo único de lo que depende todo lo demás.
4. Crear el repositorio con la estructura, proteger main y abrir el tablero con las tareas de la semana 1.